

Project Lab

Airport Database Management System

1.2 Table of Contents

1.1 Students Informations	1
1.2 Table of Contents	2
1.3 Abstract	4
1.4 Introduction	5
1.5 The objectives of the Airport Database Managements System Include	6
1.6 Number of Tables	7
1 st Airplanes Tabel.....	8
Restrictions	8
Schema Table	9
2 nd Airport Tabel.....	10
Restrictions	10
Schema Table	11
3 rd AirSurve Tabel	12
Restrictions	12
Links to Other Tables	13
Schema Table	13
4 th Baggage table.....	14
Restrictions	14
Links to Other Tables	15
Schema Table	15
5 th Employess Table	16
Restrictions	16
Schema Table	17
6 th Flights table.....	18
Restrictions	18
Links to Other Tables	19
Schema Table	19
7 TH Maintenance Table.....	20
Restrictions	20
Links to Other Tables	21
Schema Table	21

8 TH Reservations Table	22
Restrictions	22
Links to Other Tables	23
Schema Table	23
9 th Travelers Table	24
Restrictions	24
Links to Other Tables	25
Schema Table	25
1.7 Structure of the Database	26
1.8 Command to Teset Database	31

1.3 Abstract

Airport Database Management System

Our project talks about airport database management to simplify and improve various processes within the airport environment. The system will provide functions such as passenger and flight management, baggage tracking, aircraft scheduling, etc. Through effective data management and integration, the system aims to enhance operational efficiency, improve passenger experience and ensure better utilization of resources Inside the airport.

Include :

- Reservations
- Airports
- Air Control
- Maintenance
- Airplanes
- Trips
- travelers
- staff
- luggage

1.4 Introduction

Airport database management involves the organization and maintenance of data related to various aspects of airport operations, including flights, passengers, aircraft, crew, scheduling, maintenance, and security. This system aims to streamline airport operations, improve efficiency, ensure safety, and enhance the overall passenger experience.

By centralizing and integrating data from various airport functions, a database management system enhances coordination among different departments and stakeholders. It facilitates real-time communication, reduces errors, minimizes delays, and enhances overall operational performance.

Overall, an effective airport database management system plays a crucial role in ensuring the smooth and efficient functioning of airports, ultimately contributing to a safe and pleasant travel experience for passengers and efficient operations for airlines and airport authorities .

1.5 The objectives of the Airport Database Managements System Include :

1. **Accurate and Timely Information:** Ensure that accurate and up-to-date information regarding flights, passengers, baggage, personnel, and resources is readily available to airport staff and stakeholders.
2. **Improved Operational Efficiency:** Streamline airport operations by automating routine tasks, optimizing resource allocation, and facilitating better coordination between different departments.
3. **Enhanced Passenger Experience:** Provide passengers with seamless travel experiences by facilitating smoother check-in processes, timely flight information, and efficient baggage handling.
4. **Effective Resource Utilization:** Enable airport authorities to make informed decisions regarding resource allocation, such as gates, terminals, ground handling equipment, and staff, to maximize efficiency and minimize costs.
5. **Compliance and Security:** Ensure compliance with aviation regulations and enhance airport security by accurately tracking passenger and baggage movements and maintaining detailed records of airport activities.
6. **Scalability and Flexibility:** Design the database system to be scalable and adaptable to accommodate future growth, technological advancements, and evolving airport requirements.

1.6 Number of tables:

Our database contains 9 tables (airline database), each of which serves a specific role within the database system, linking various aspects of airline operations.

1. **Airplanes Table:** This table contains records of airplanes, including details like model, capacity, range, and status.
2. **Airport Table:** This table stores information about airports, such as location, capacity, available services, and operational hours.
3. **AirSurve Table:** this table tracks air surveillance data, possibly including flight paths, airspace monitoring, and weather conditions.
4. **Baggage Table:** This table manages baggage information, tracking baggage check-ins, transfers, and claims.
5. **Employees Table:** This table lists airline employees, their roles, qualifications, and work schedules.
6. **Flights Table:** This table contains flight schedules, including departure and arrival times, routes, and aircraft assignments.
7. **Maintenance Table:** This table tracks maintenance activities for aircraft, detailing inspections, repairs, and parts replacements.
8. **Reservations Table:** This table manages flight bookings, including passenger information, seat assignments, and payment details.
9. **Travelers Table:** This table records information on travelers, possibly including frequent flyer data, travel history, and preferences.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_miniproject |
+-----+
| airplanes              |
| airportemployees       |
| airports               |
| airsurveillance        |
| baggage                |
| flights                |
| maintenance            |
| reservations           |
| travelers               |
+-----+
9 rows in set (0.00 sec)
```

1st Airplanes Tabel

This table is the "Airplanes" table from the airline database. It is designed to hold information about the different aircraft that are part of the airline's fleet. Below is the detailed explanation of the table along with its constraints and schema:

- **AirplaneID:** This is the primary key for the table, ensuring each entry for an airplane is unique. The data type is INT, which allows for numeric identifiers that are integers. As a primary key, it will have a uniqueness restriction, meaning no two airplanes can have the same 'AirplaneID'.
- **Model:** This VARCHAR column holds the model name of the airplane. The VARCHAR data type is chosen for text strings, and here it likely allows for a variable length to accommodate different model names. There's no explicit unique restriction, so it's possible for multiple entries to have the same model name.
- **Manufacturer:** This VARCHAR field represents the company that manufactured the airplane. Similar to Model, it's text-based and can include different manufacturers' names. There's no unique restriction indicated here as well.
- **MaxCapacity:** An INT field indicating the maximum seating capacity of the airplane. As an integer, it represents a whole number. There are no restrictions indicated that limit the values this field can hold, aside from the natural restriction that capacity should be a positive integer.
- **YearOfManufacture:** This is also an INT field that records the year the airplane was manufactured. It must also be a positive integer and typically would represent a year in the recent past since airplanes have a certain operational lifespan after manufacturing.

Restrictions

- All fields are marked as **NOT NULL**, which is a restriction ensuring that records cannot be created with any missing information; each record must have a value for every column.
- **"AirplaneID"** is the only field explicitly marked as a primary key, which requires that each value in this column must be unique and not null.

Schema Table

The schema for the Airplanes table, including the data types and restrictions, can be summarized as follows:

Column Name	Data Type	Restrictions
AirplaneID	INT	PRIMARY KEY, NOT NULL
Model	VARCHAR	NOT NULL
Manufacturer	VARCHAR	NOT NULL
MaxCapacity	INT	NOT NULL
YearOfManufacture	INT	NOT NULL

The table after execution SELECT command:

AirplaneID	Model	Manufacturer	MaxCapacity	YearOfManufacture
801	Emirates 747	Emirates	360	2001
802	Saudia S380	Saudia	150	2007
803	Air arabia 172	Air arabia	240	1999
804	Gluf Air G175	Gluf Air	180	2003
805	Flydubai 300	Flydubai	160	2001
806	Qatar Airways Q-130	Qatar Airways	190	2004
807	FlyNas N650	FlyNas	180	2008

7 rows in set (0.01 sec)

2nd Airport Tabel

The `Airports` table stores data about various airports. Each record contains details about an individual airport. Here's a detailed analysis of the table:

- **AirportsID:** This column is an auto-incrementing integer that serves as the primary key. It uniquely identifies each airport entry and will automatically increase with each new record, ensuring uniqueness and not allowing null values.
- **airport_name:** A variable character string column with a maximum length of 255 characters. It holds the name of the airport. The field does not allow null values but does not need to be unique, as different records might potentially share the same airport name in case of naming after the same person or entity.
- **geographical_location:** Another variable character string with a maximum length of 255 characters. It stores the physical location of the airport, including city and country. It is not null, meaning every airport record must have a location specified.
- **airport_code:** A variable character string, likely with a standard length for airport codes (typically 3 characters for IATA codes). This field is indexed, which means there is a performance optimization for searching on this field, and while it is not specified as unique in the schema creation, typically, airport codes are unique in real- world applications.

Restrictions

- **NOT NULL:** All columns are specified as NOT NULL, which is a restriction that ensures that no record can be entered into the database without values in all of these fields.
- **PRIMARY KEY:** The `AirportsID` is the primary key, which adds a uniqueness constraint to this column.
- **INDEX:** The `airport\_code` is indexed. While this is not a constraint, it does suggest that this field is important for searching and may be used frequently to reference the table, implying a unique characteristic.

Schema Table

Here’s the schema for the Airports table, listing columns, data types, and restrictions:

Column Name	Data Type	Restrictions
AirportsID	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
airport_name	VARCHAR(255)	NOT NULL
geographical_location	VARCHAR(255)	NOT NULL
airport_code	VARCHAR(3)	INDEX, NOT NULL

The table after execution SELECT command:

AirportsID	airport_name	geographical_location	airport_code
1	King Abdulaziz International Airport	Jeddah, Saudi Arabia	JED
2	King Fahd International Airport	Dammam, Saudi Arabia	DMM
3	Dubai International Airport	Dubai, UAE	DXB
4	Hamad International Airport	Doha, Qatar	DOH
5	Cairo International Airport	Cairo, Egypt	CAI
6	King Khalid International Airport	Riyadh, Saudi Arabia	RUH
7	Abu Dhabi International Airport	Abu Dhabi, UAE	AUH

7 rows in set (0.00 sec)

3rd AirSurve Tabel

The ``AirSurveillance`` table is designed to keep track of surveillance activities related to flights. Here's a detailed breakdown of the table:

- **AirSurveillanceID:** Serves as the primary key of the table with an ``INT`` data type and is auto-incrementing. It uniquely identifies each surveillance record and cannot be null.
- **FlightNo:** A ``VARCHAR`` column that stores the flight number. It is likely to be a foreign key that references the ``Flights`` table, creating a relationship between surveillance entries and specific flights.
- **Surveillance_datetime:** A ``DATETIME`` column that records the exact date and time when the surveillance activity took place. This field does not allow null values, ensuring that every surveillance record has a timestamp.
- **Surveillance_results:** A ``TEXT`` column providing details on the results of the surveillance. This could be variable in length, which is why ``TEXT`` is used, and it does not allow null values.
- **Controller:** A ``VARCHAR(50)`` column that notes the name of the controller in charge of the surveillance. The maximum length of 50 characters is allowed, and it also cannot be null.

Restrictions

- **NOT NULL:** All columns have the NOT NULL restriction, ensuring that records cannot be inserted into this table without values for each of these fields.
- **PRIMARY KEY:** The ``AirSurveillanceID`` column has a PRIMARY KEY constraint, which ensures the uniqueness of each record in the table.
- **FOREIGN KEY:** The ``FlightNo`` column has a FOREIGN KEY relationship with the ``Flights`` table, connecting surveillance data to specific flights. This implies that each ``FlightNo`` must correspond to an existing record in the ``Flights`` table.

Links to Other Tables

- *Flights Table:* The `FlightNo` column creates a direct link to the `Flights` table, meaning for each surveillance activity logged, there must be a corresponding flight record.

Schema Table

The schema for the `AirSurveillance` table, which details the columns, their data types, and restrictions, would look like this:

Column Name	Data Type	Restrictions
AirSurveillanceID	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
FlightNo	VARCHAR	FOREIGN KEY, NOT NULL
Surveillance_datetime	DATETIME	NOT NULL
Surveillance_results	TEXT	NOT NULL
Controller	VARCHAR(50)	NOT NULL

The table after execution SELECT command:

AirSurveillanceID	FlightNo	Surveillance_datetime	Surveillance_results	Controller
1	P7BPTV	2023-09-15 17:30:00	Weather advisory	Mishel Fahd
2	DK8SM4	2023-09-16 14:45:00	No issues reported	Mishel Fahd
3	PWM4XC	2023-09-18 18:00:00	Engine malfunction	Mishel Fahd
4	OW2V9M	2023-09-19 15:30:00	Normal operation	Mishel Fahd
5	Q7V3SK	2023-09-22 12:45:00	Weather clearance	Mishel Fahd

5 rows in set (0.00 sec)

4th Baggage table

This table is the baggage table, which is used to track the baggage associated with flights and passengers. Below is a detailed analysis of the table:

- **baggageID:** This is the primary key for the table with an `INT` data type and is set to auto-increment. It uniquely identifies each piece of baggage and does not allow null values.
- **weight:** A `DECIMAL` data type column that stores the weight of the baggage. There is no explicit NOT NULL restriction provided in the creation script, but it is common to require a weight for each baggage record.
- **FlightNo:** A `VARCHAR(50)` column that acts as a foreign key referencing the `Flights` table, associating each baggage with a specific flight. This field does not allow null values.
- **TravelerNo:** A `VARCHAR(20)` column that also acts as a foreign key referencing the `Travelers` table, linking each baggage item to a specific traveler. It is also a non- null field.

Restrictions

- **NOT NULL:** The `baggageID`, `FlightNo`, and `TravelerNo` columns are specified as NOT NULL, ensuring that no record can exist without these essential pieces of information.
- **PRIMARY KEY:** The `baggageID` column is the primary key, imposing a uniqueness constraint and not allowing nulls.
- **FOREIGN KEYS:** Both `FlightNo` and `TravelerNo` are foreign keys, which means that their values must match entries in the respective `Flights` and `Travelers` tables. This ensures referential integrity within the database.

Links to Other Tables

- **Flights Table:** The **FlightNo** column establishes a link to the **Flights** table, tying the baggage to specific flights.
- **Travelers Table:** The **TravelerNo** column links each baggage record to a specific traveler in the **Travelers** table.

Schema Table

The schema for the Baggage table, detailing columns, data types, and constraints, is as follows:

Column Name	Data Type	Restrictions
baggageID	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
weight	DECIMAL	(Assumed NOT NULL)
FlightNo	VARCHAR(50)	FOREIGN KEY, NOT NULL
TravelerNo	VARCHAR(20)	FOREIGN KEY, NOT NULL

The table after execution SELECT command:

baggageID	weight	FlightNo	TravelerNO
1211	20.50	P7BPTV	T101
1221	15.20	DK8SM4	T102
1231	18.70	L9W3RG	T103
1241	22.10	PWM4XC	T104
1251	16.80	OW2V9M	T105
1261	19.30	Q7V3SK	T106
1271	7.20	OD3VY7	T107

7 rows in set (0.00 sec)

5th Employess Table

The `AirportEmployees` table is structured to maintain records of employees working at an airport. Here's a detailed explanation of the table from the provided images:

- **employeeID:** This is an auto-incrementing integer that serves as the primary key. It uniquely identifies each employee record and is not allowed to be null.
- **name:** A variable character string column with a maximum length of 100 characters, storing the name of the employee. It does not allow null values.
- **position:** Another `VARCHAR(100)` column that specifies the employee's job title or position at the airport, also not null.
- **department:** This `VARCHAR(100)` field defines the department within the airport in which the employee works. Like the other `VARCHAR` fields, it does not permit nulls.
- **shift:** A `VARCHAR(50)` column indicating the shift pattern the employee works, such as Day, Night, or Rotating. It is required for each record, thus not null.
- **salary:** A `DECIMAL(10, 2)` field that records the salary of the employee. The decimal type is used to accommodate monetary values which can have two digits after the decimal point. This field is also not null.

Restrictions

- **NOT NULL:** Each column has the NOT NULL constraint, which requires that every record has a value for each column when inserted into the table.
- **PRIMARY KEY:** The `employeeID` has a PRIMARY KEY constraint ensuring the uniqueness of each record.

Schema Table

Here’s the schema for the AirportEmployees table, including the columns, their data types, and restrictions:

Column Name	Data Type	Restrictions
employeeID	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
name	VARCHAR(100)	NOT NULL
position	VARCHAR(100)	NOT NULL
department	VARCHAR(100)	NOT NULL
shift	VARCHAR(50)	NOT NULL
salary	DECIMAL(10,2)	NOT NULL

The table after execution SELECT command:

employeeID	name	position	department	shift	salary
1	Ahmed Ali	Security	Security	Day	4500.00
2	Abdulaziz Omr	Pilot	Flight Crew	Night	90000.00
3	Mishel Fahd	Air Traffic Controller	Air Traffic Control	Rotating	60000.00
4	Turki Mohammed	Customer Service Agent	Passenger Services	Evening	4000.00
5	Walied Talal	Baggage Handler	Ground Operations	Morning	3500.00
6	Mohammed Fahd	Airport Manager	Administration	Day	10000.00
7	Jamel Hassen	Maintenance Worker	Facilities	Rotating	55000.00

7 rows in set (0.00 sec)

6th Flights table

The `Flights` table contains information about airline flights. Each row represents a specific flight with various details. Here's a detailed analysis of the table:

- **FlightNo:** A `VARCHAR(50)` field that serves as the primary key. This column holds the flight number and cannot contain null values. Each flight number must be unique.
- **AirplaneID:** An `INT` field that likely represents a foreign key reference to the `Airplanes` table. It identifies which airplane is used for the flight and must match an existing `AirplaneID` in the `Airplanes` table.
- **AirplaneType:** A `VARCHAR(50)` field that describes the type of airplane. This field cannot be null but does not necessarily need to be unique.
- **DepartureTime:** A `TIME` field indicating when the flight departs. It cannot be null, ensuring that every flight entry has a departure time.
- **ArrivalTime:** A `TIME` field that represents the expected arrival time of the flight. It also cannot be null.
- **FlightDuration:** A `VARCHAR(50)` field providing the expected duration of the flight, such as "3 Hours". It cannot be null.
- **DestinationAirport:** A `VARCHAR(50)` field that uses an airport code to reference the `Airports` table. This is also a foreign key that links each flight to its destination airport and cannot be null.

Restrictions

- **NOT NULL:** All fields are required to have values; none can be left empty when records are inserted.
- **PRIMARY KEY:** The `FlightNo` column has a PRIMARY KEY constraint, enforcing uniqueness across all entries.
- **FOREIGN KEYS:** There are two foreign key constraints:
 - **`AirplaneID`** references the `AirplaneID` in the `Airplanes` table, linking a flight to a specific airplane.
 - **`DestinationAirport`** references the `airport\_code` in the `Airports` table, establishing a relationship between a flight and its destination airport.

Links to Other Tables

- **Airplanes Table:** The `AirplaneID` links each flight to a specific airplane.
- **Airports Table:** The `DestinationAirport` creates a reference to a specific airport, indicating where the flight is scheduled to land.

Schema Table

Here's the schema for the Flights table, with columns, data types, and restrictions:

Column Name	Data Type	Restrictions
FlightNo	VARCHAR(50)	PRIMARY KEY, NOT NULL
AirplaneID	INT	FOREIGN KEY, NOT NULL
AirplaneType	VARCHAR(50)	NOT NULL
DepartureTime	TIME	NOT NULL
ArrivalTime	TIME	NOT NULL
FlightDuration	VARCHAR(50)	NOT NULL
DestinationAirport	VARCHAR(50)	FOREIGN KEY (to Airports.airport_code), NOT NULL

The table after execution SELECT command:

FlightNo	AirplaneID	AirplaneType	DepartmentTime	ArrivalTime	FlightDuration	DestinationAirport
DK8SM4	802	Airbus	13:30:00.00	16:30:00.00	3 Hours	DMM
L9W3RG	803	Airbus	10:55:00.00	12:00:00.00	1 Hours	DXB
OD3VY7	807	Airbus	17:20:00.00	19:30:00.00	2 Hours	AUH
OW2V9M	805	Airbus	11:50:00.00	19:05:00.00	6 Hours	CAI
P7BPTV	801	Airbus	16:55:00.00	18:55:00.00	2 Hours	JED
PWM4XC	804	Airbus	26:40:00.00	18:55:00.00	2 Hours	DOH
Q7V3SK	806	Airbus	11:00:00.00	15:10:00.00	4 Hours	RUH

7 rows in set (0.00 sec)

7TH Maintenance Table

The `Maintenance` table records details about maintenance activities conducted on airplanes. Here's the detailed information about the table:

- **MaintenanceID:** This is an auto-incrementing integer that serves as the primary key, uniquely identifying each maintenance record. It cannot be null.
- **AirplaneID:** An integer field that likely references an airplane in the `Airplanes` table. This is a foreign key that establishes a relationship between maintenance records and specific airplanes.
- **maintenance_datetime:** A `DATETIME` column that records the date and time when the maintenance occurred.
- **maintenance_type:** A `VARCHAR(50)` field that describes the type of maintenance performed.
- **responsible_employee:** A `VARCHAR(100)` column that stores the name of the employee responsible for the maintenance.

Restrictions

- **NOT NULL:** All columns are set to not allow null values, meaning every maintenance record must have complete information upon entry.
- **PRIMARY KEY:** `MaintenanceID` is the primary key and thus must be unique and not null.
- **FOREIGN KEY:** The `AirplaneID` field is a foreign key that must match an existing `AirplaneID` in the `Airplanes` table, enforcing referential integrity.

Links to Other Tables

- **Airplanes Table:** The `AirplaneID` creates a direct link to the `Airplanes` table, associating maintenance records with specific airplanes.

Schema Table

The schema for the Maintenance table, detailing the columns, data types, and restrictions, would be as follows:

Column Name	Data Type	Restrictions
MaintenanceID	INT	PRIMARY KEY, AUTO_INCREMENT, NOT NULL
AirplaneID	INT	FOREIGN KEY, NOT NULL
maintenance_datetime	DATETIME	NOT NULL
maintenance_type	VARCHAR(50)	NOT NULL
responsible_employee	VARCHAR(100)	NOT NULL

The table after execution SELECT command:

MaintenanceID	AirplaneID	maintenance_datetime	maintenance_type	responsible_employee
1	801	2022-10-01 14:30:00	Emergency engine repair	Jamel Hassn
2	802	2019-12-22 10:15:00	Navigation system update	Jamel Hassn
3	802	2024-02-03 12:50:00	Hydraulic fluid change	Jamel Hassn
4	803	2020-11-04 09:40:00	Cabin lighting inspection	Jamel Hassn
5	804	2018-01-15 13:00:00	Emergency landing gear replacement	Jamel Hassn
6	805	2024-02-06 08:25:00	Avionics software patch	Jamel Hassn
7	806	2024-07-27 11:10:00	Wing flap adjustment	Jamel Hassn
8	807	2009-03-09 10:05:00	Routine engine check	Jamel Hassn

8 rows in set (0.00 sec)

8TH Reservations Table

The `RESERVATIONS` table is part of a database for managing flight bookings. Here's an in-depth look at the table:

- **BookingNO:** This is a `VARCHAR(50)` column and serves as the primary key, which means it must contain unique values for each booking.
- **FlightNo:** A `VARCHAR(50)` field that likely represents the flight number associated with the booking. It is a foreign key that references the `Flights` table.
- **FlightHistory:** A `DATE` field that records the date associated with the flight history.
The date does not include a time component.
- **BookingStatus:** A `TEXT` column that describes the status of the booking, such as "Confirmed", "Pending", "Cancelled", or "Completed".

Restrictions

- **NOT NULL:** All fields are marked as NOT NULL, ensuring that no record can be created without a booking number, flight number, flight history date, or booking status.
- **PRIMARY KEY:** `BookingNO` has a PRIMARY KEY constraint, which enforces uniqueness across the dataset.
- **FOREIGN KEY:** `FlightNo` has a FOREIGN KEY relationship with the `Flights` table, which ensures that each booking is linked to a valid flight.

Links to Other Tables

- **Flights Table:** The `FlightNo` field links each reservation to a specific flight in the `Flights` table, allowing the database to maintain relational integrity and connect bookings to their corresponding flight details.

Schema Table

The schema for the RESERVATIONS table, outlining the columns, their data types, and restrictions, is as follows:

Column Name	Data Type	Restrictions
BookingNO	VARCHAR(50)	PRIMARY KEY, NOT NULL
FlightNo	VARCHAR(50)	FOREIGN KEY, NOT NULL
FlightHistory	DATE	NOT NULL
BookingStatus	TEXT	NOT NULL

The table after execution SELECT command:

BookingNO	FlightNo	FlightHistory	BookingStatus
B0001	P7BPTV	2023-09-15	Confirmed
B0002	DK8SM4	2023-09-16	Pending
B0003	L9W3RG	2023-09-17	Cancelled
B0004	PWM4XC	2023-09-18	Completed
B0005	OW2V9M	2023-09-19	Pending
B0006	Q7V3SK	2023-09-22	Confirmed
B0007	OD3VY7	2023-09-25	Cancelled

7 rows in set (0.01 sec)

9th Travelers Table

The `Travelers` table is set up to store information about individuals who have made travel reservations. Below is a detailed analysis of the table:

- **TravelerNO:** This is a `VARCHAR(50)` column that serves as the primary key of the table. It holds a unique identifier for each traveler.
- **bookingNO:** A `VARCHAR(50)` field that contains the booking number associated with the traveler. It has a foreign key constraint linking it to the `RESERVATIONS` table.
- **FirstName:** This `VARCHAR(50)` column stores the first name of the traveler.
- **LastName:** A `VARCHAR(50)` field for storing the last name of the traveler.
- **PassportNumber:** A `VARCHAR(20)` column that holds the passport number of the traveler. This is typically unique to each individual.
- **DateOfBirth:** A `DATE` column for the traveler's date of birth.
- **Nationality:** A `VARCHAR(50)` column that records the nationality of the traveler.
- **PhoneNumber:** A `VARCHAR(50)` column that contains the phone number of the traveler.

Restrictions

- **NOT NULL:** All columns are constrained with NOT NULL, indicating that each field must be filled out; no null values are permitted for any record.
- **PRIMARY KEY:** `TravelerNO` has the PRIMARY KEY constraint, necessitating that each value in this column be unique.
- **FOREIGN KEY:** `bookingNO` is a FOREIGN KEY that references the
`RESERVATIONS` table, ensuring the linkage of travelers to their respective booking records.

Links to Other Tables

- **RESERVATIONS Table:** The `bookingNO` field is the link to the `RESERVATIONS` table, allowing for the connection between a traveler and their flight booking.

Schema Table

The schema for the Travelers table, which details the columns, their data types, and restrictions, would be as follows:

Column Name	Data Type	Restrictions
TravelerNO	VARCHAR(50)	PRIMARY KEY, NOT NULL
bookingNO	VARCHAR(50)	FOREIGN KEY, NOT NULL
FirstName	VARCHAR(50)	NOT NULL
LastName	VARCHAR(50)	NOT NULL
PassportNumber	VARCHAR(20)	NOT NULL
DateOfBirth	DATE	NOT NULL
Nationality	VARCHAR(50)	NOT NULL
PhoneNumber	VARCHAR(50)	NOT NULL

The table after execution SELECT command:

TravelerNO	bookingNO	FirstName	LastName	PassportNumber	DateOfBirth	Nationality	PhoneNumber
T101	B0001	Rose	Nife	AB123456	1990-01-22	SAUDI	0539847599
T102	B0002	Rana	Abdullah	RA1212156	1999-10-12	SAUDI	0567823465
T103	B0003	Sara	Saad	SA983456	2000-02-15	SAUDI	0587234566
T104	B0004	Mada	Jamel	MA116836	1996-07-11	SAUDI	0599821345
T105	B0005	Rahaf	Ali	RJ987345	2002-06-29	SAUDI	058763211234
T106	B0006	Jana	Mohamed	CD789012	2001-08-25	SAUDI	0598773214
T107	B0007	Naif	Ahmed	FG182903	1997-04-23	SAUDI	0518254930

7 rows in set (0.00 sec)

1.8 Structure of the Database

1) Airplanes table :

```
mysql> USE MiniProject;
Database changed
mysql> CREATE TABLE Airplanes (
  ->   AirplaneID INT PRIMARY KEY,
  ->   Model VARCHAR(50) NOT NULL,
  ->   Manufacturer VARCHAR(50) NOT NULL,
  ->   MaxCapacity INT NOT NULL,
  ->   YearOfManufacture INT NOT NULL);
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> INSERT INTO Airplanes (AirplaneID, Model, Manufacturer, MaxCapacity, YearOfManufacture)
  -> VALUES
  ->   ( 801, 'Emirates 747', 'Emirates', 360, 2001),
  ->   ( 802, 'Saudia S380', 'Saudia', 150, 2007),
  ->   ( 803, 'Air arabia 172', 'Air arabia', 240, 1999),
  ->   ( 804, 'Gluf Air G175', 'Gluf Air', 180, 2003),
  ->   ( 805, 'Flydubai 300', 'Flydubai', 160, 2001),
  ->   ( 806, 'Qatar Airways Q-130 ', 'Qatar Airways', 190, 2004),
  ->   ( 807, 'FlyNas N650', 'FlyNas', 180, 2008);
Query OK, 7 rows affected (0.02 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

2) AirportEmployee table :

```
mysql> CREATE TABLE AirportEmployees (
  ->   employeeID INT AUTO_INCREMENT PRIMARY KEY,
  ->   name VARCHAR(100) NOT NULL,
  ->   position VARCHAR(100) NOT NULL,
  ->   department VARCHAR(100) NOT NULL,
  ->   shift VARCHAR(50) NOT NULL,
  ->   salary DECIMAL(10, 2) NOT NULL
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql>
mysql> INSERT INTO AirportEmployees (name, position, department, shift, salary) VALUES
  ->   ('Ahmed Ali', 'Security', 'Security', 'Day', 4500),
  ->   ('Abdulaziz Omr', 'Pilot', 'Flight Crew', 'Night', 90000),
  ->   ('Mishel Fahd', 'Air Traffic Controller', 'Air Traffic Control', 'Rotating', 60000),
  ->   ('Turki Mohammed', 'Customer Service Agent', 'Passenger Services', 'Evening', 4000),
  ->   ('Walied Talal', 'Baggage Handler', 'Ground Operations', 'Morning', 3500),
  ->   ('Mohammed Fahd', 'Airport Manager', 'Administration', 'Day', 10000),
  ->   ('Jamel Hassan', 'Maintenance Worker', 'Facilities', 'Rotating', 55000);
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

3) Airports table :

```
mysql> CREATE TABLE Airports (
->   AirportsID INT AUTO_INCREMENT PRIMARY KEY,
->   airport_name VARCHAR(255),
->   geographical_location VARCHAR(255),
->   airport_code VARCHAR(3),
->   INDEX(airport_code)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Airports (airport_name, geographical_location, airport_code)
-> VALUES
-> ('King Abdulaziz International Airport', 'Jeddah, Saudi Arabia', 'JED'),
-> ('King Fahd International Airport', 'Dammam, Saudi Arabia', 'DMM'),
-> ('Dubai International Airport', 'Dubai, UAE', 'DXB'),
-> ('Hamad International Airport', 'Doha, Qatar', 'DOH'),
-> ('Cairo International Airport', 'Cairo, Egypt', 'CAI'),
-> ('King Khalid International Airport', 'Riyadh, Saudi Arabia', 'RUH'),
-> ('Abu Dhabi International Airport', 'Abu Dhabi, UAE', 'AUH');
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

4) AirSurveillance table :

```
mysql> CREATE TABLE AirSurveillance (
->   AirSurveillanceID INT AUTO_INCREMENT PRIMARY KEY,
->   FlightNo VARCHAR(10) NOT NULL,
->   FOREIGN KEY (FlightNo) REFERENCES flights(FlightNo),
->   Surveillance_datetime DATETIME NOT NULL,
->   Surveillance_results TEXT,
->   Controller VARCHAR(50) NOT NULL
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql>
mysql> INSERT INTO AirSurveillance (FlightNo, Surveillance_datetime, Surveillance_results, Controller)
-> VALUES
-> ('P7BPTV', '2023-09-15 17:30:00', 'Weather advisory', 'Mishel Fahd'),
-> ('DK8SM4', '2023-09-16 14:45:00', 'No issues reported', 'Mishel Fahd'),
-> ('PMM4XC', '2023-09-18 18:00:00', 'Engine malfunction', 'Mishel Fahd'),
-> ('OM2V9M', '2023-09-19 15:30:00', 'Normal operation', 'Mishel Fahd'),
-> ('Q7V3SK', '2023-09-22 12:45:00', 'Weather clearance', 'Mishel Fahd');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

5) Baggage table :

```
mysql> CREATE TABLE Baggage (  
-> baggageID INT PRIMARY KEY,  
-> weight DECIMAL(10,2) NOT NULL,  
-> FlightNo VARCHAR(50) NOT NULL,  
-> FOREIGN KEY (FlightNo) REFERENCES flights(FlightNo),  
-> TravelerNO VARCHAR(20) NOT NULL,  
-> FOREIGN KEY (TravelerNO) REFERENCES Travelers(TravelerNO)  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> INSERT INTO Baggage (baggageID, weight, FlightNo, TravelerNO) VALUES  
-> (1211, 20.5, 'P7BPTV', 'T101'),  
-> (1221, 15.2, 'DK8SM4', 'T102'),  
-> (1231, 18.7, 'L9W3RG', 'T103'),  
-> (1241, 22.1, 'PMM4XC', 'T104'),  
-> (1251, 16.8, 'OW2V9M', 'T105'),  
-> (1261, 19.3, 'Q7V3SK', 'T106'),  
-> (1271, 7.2, 'OD3VY7', 'T107');  
Query OK, 7 rows affected (0.01 sec)  
Records: 7 Duplicates: 0 Warnings: 0
```

6) Flight table :

```
mysql> CREATE TABLE flights (  
-> FlightNo varchar(50) NOT NULL PRIMARY KEY,  
-> AirplaneID INT,  
-> FOREIGN KEY (AirplaneID) REFERENCES Airplanes(AirplaneID),  
-> AirplaneType varchar(50) not null,  
-> DepartmentTime time(2) not null,  
-> ArrivallTime time(2) not null,  
-> FlightDuration varchar(50) not null,  
-> DestinationAirport varchar(50) not null,  
-> FOREIGN KEY (DestinationAirport) REFERENCES Airports(airport_code)  
-> );  
Query OK, 0 rows affected (0.05 sec)  
  
mysql>  
mysql> INSERT INTO flights VALUES  
-> ("P7BPTV", 801, "Airbus", "16:55", "18:55", "2 Hours", "JED"),  
-> ("DK8SM4", 802, "Airbus", "13:30", "16:30", "3 Hours", "DMM"),  
-> ("L9W3RG", 803, "Airbus", "10:55", "12:00", "1 Hours", "DXB"),  
-> ("PMM4XC", 804, "Airbus", "26:40", "18:55", "2 Hours", "DOH"),  
-> ("OW2V9M", 805, "Airbus", "11:50", "19:05", "6 Hours", "CAI"),  
-> ("Q7V3SK", 806, "Airbus", "11:00", "15:10", "4 Hours", "RUH"),  
-> ("OD3VY7", 807, "Airbus", "17:20", "19:30", "2 Hours", "AUH");  
Query OK, 7 rows affected (0.01 sec)  
Records: 7 Duplicates: 0 Warnings: 0
```

7) Maintenance table :

```
mysql> CREATE TABLE Maintenance (
-> MaintenanceID INT AUTO_INCREMENT PRIMARY KEY,
-> AirplaneID INT NOT NULL,
-> FOREIGN KEY (AirplaneID) REFERENCES Airplanes(AirplaneID),
-> maintenance_datetime DATETIME,
-> maintenance_type VARCHAR(50) NOT NULL,
-> responsible_employee VARCHAR(100) NOT NULL
-> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Maintenance (AirplaneID, maintenance_datetime, maintenance_type, responsible_employee)
-> VALUES
-> (801, '2022-10-01 14:30:00', 'Emergency engine repair', 'Jamel Hassn'),
-> (802, '2019-12-22 10:15:00', 'Navigation system update', 'Jamel Hassn'),
-> (802, '2024-02-03 12:50:00', 'Hydraulic fluid change', 'Jamel Hassn'),
-> (803, '2020-11-04 09:40:00', 'Cabin lighting inspection', 'Jamel Hassn'),
-> (804, '2018-01-15 13:00:00', 'Emergency landing gear replacement', 'Jamel Hassn'),
-> (805, '2024-02-06 08:25:00', 'Avionics software patch', 'Jamel Hassn'),
-> (806, '2024-07-27 11:10:00', 'Wing flap adjustment', 'Jamel Hassn'),
-> (807, '2009-03-09 10:05:00', 'Routine engine check', 'Jamel Hassn');
Query OK, 8 rows affected (0.01 sec)
Records: 8 Duplicates: 0 Warnings: 0
```

8) Reservations table :

```
mysql> CREATE TABLE RESERVATIONS (
-> BookingNO varchar(50) NOT NULL PRIMARY KEY,
-> FlightNo varchar(50) NOT NULL,
-> FlightHistory DATE NOT NULL,
-> BookingStatus TEXT NOT NULL,
-> FOREIGN KEY (FlightNo) REFERENCES flights(FlightNo)
-> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO RESERVATIONS (BookingNO , FlightNo, FlightHistory, BookingStatus) VALUES
-> ('B0001', 'P7BPTV', '2023-09-15', 'Confirmed'),
-> ('B0002', 'DK8SM4', '2023-09-16', 'Pending'),
-> ('B0003', 'L9W3RG', '2023-09-17', 'Cancelled'),
-> ('B0004', 'PWN4XC', '2023-09-18', 'Completed'),
-> ('B0005', 'OW2V9M', '2023-09-19', 'Pending'),
-> ('B0006', 'Q7V3SK', '2023-09-22', 'Confirmed'),
-> ('B0007', 'OD3VY7', '2023-09-25', 'Cancelled');
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

9) Travelers table :

```
mysql> CREATE TABLE Travelers(
-> TravelerNO varchar(50) NOT NULL PRIMARY KEY,
-> bookingNO varchar(50) NOT NULL ,
-> FOREIGN KEY (bookingNO) REFERENCES RESERVATIONS(bookingNO),
-> FirstName VARCHAR(50)NOT NULL,
-> LastName VARCHAR(50)NOT NULL,
-> PassportNumber VARCHAR(20)NOT NULL,
-> DateOfBirth DATE NOT NULL,
-> Nationality VARCHAR(50)NOT NULL,
-> PhoneNumber VARCHAR(15)NOT NULL);
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO Travelers(TravelerNO, bookingNO, FirstName, LastName, PassportNumber, DateOfBirth, Nationality, PhoneNumber)
-> VALUES
-> ('T101','B0001', 'Rose', 'Nife', 'AB123456', '1990-01-22', 'SAUDI', '0539847599'),
-> ('T102','B0002', 'Rana', 'Abdullah', 'RA1212156', '1999-10-12', 'SAUDI', '0567823465'),
-> ('T103','B0003', 'Sara', 'Saad', 'SA983456', '2000-02-15', 'SAUDI', '0587234566'),
-> ('T104','B0004', 'Mada', 'Jamel', 'MA116836', '1996-07-11', 'SAUDI', '0599821345'),
-> ('T105','B0005', 'Rahaf', 'Ali', 'RJ987345', '2002-06-29', 'SAUDI', '058763211234'),
-> ('T106','B0006', 'Jana', 'Mohamed', 'CD789012', '2001-08-25', 'SAUDI', '0598773214'),
-> ('T107','B0007', 'Naif', 'Ahmed', 'FG182903', '1997-4-23', 'SAUDI', '0518254930');
Query OK, 7 rows affected (0.01 sec)
Records: 7 Duplicates: 0 Warnings: 0
```

1.9 Command to Test Database

```
mysql> desc AirportEmployees;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| employeeID | int           | NO   | PRI | NULL    | auto_increment |
| name       | varchar(100)  | NO   |     | NULL    |                |
| position   | varchar(100)  | NO   |     | NULL    |                |
| department | varchar(100)  | NO   |     | NULL    |                |
| shift      | varchar(50)   | NO   |     | NULL    |                |
| salary     | decimal(10,2) | NO   |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.02 sec)
```

Desc AirportEmployees

```
mysql> alter table Maintenance modify responsible_employee varchar(90);
Query OK, 8 rows affected (0.10 sec)
Records: 8 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Maintenance;
+-----+-----+-----+-----+-----+
| MaintenanceID | AirplaneID | maintenance_datetime | maintenance_type | responsible_employee |
+-----+-----+-----+-----+-----+
| 1 | 801 | 2022-10-01 14:30:00 | Emergency engine repair | Jamel Hassn |
| 2 | 802 | 2019-12-22 10:15:00 | Navigation system update | Jamel Hassn |
| 3 | 802 | 2024-02-03 12:50:00 | Hydraulic fluid change | Jamel Hassn |
| 4 | 803 | 2020-11-04 09:40:00 | Cabin lighting inspection | Jamel Hassn |
| 5 | 804 | 2018-01-15 13:00:00 | Emergency landing gear replacement | Jamel Hassn |
| 6 | 805 | 2024-02-06 08:25:00 | Avionics software patch | Jamel Hassn |
| 7 | 806 | 2024-07-27 11:10:00 | Wing flap adjustment | Jamel Hassn |
| 8 | 807 | 2009-03-09 10:05:00 | Routine engine check | Jamel Hassn |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

Alter maintenance modify responable_employees varchar(90)

SELECT * FROM maintenance

```
mysql> alter table AirportEmployees add overtime DECIMAL(10, 2);
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM AirportEmployees;
```

employeeID	name	position	department	shift	salary	overtime
1	Ahmed Ali	Security	Security	Day	4500.00	NULL
2	Abdulaziz Omr	Pilot	Flight Crew	Night	90000.00	NULL
3	Mishel Fahd	Air Traffic Controller	Air Traffic Control	Rotating	60000.00	NULL
4	Turki Mohammed	Customer Service Agent	Passenger Services	Evening	4000.00	NULL
5	Walied Talal	Baggage Handler	Ground Operations	Morning	3500.00	NULL
6	Mohammed Fahd	Airport Manager	Administration	Day	10000.00	NULL
7	Jamel Hassn	Maintenance Worker	Facilities	Rotating	55000.00	NULL

```
7 rows in set (0.00 sec)
```

Alter table AirportEmployees add overtime DECIMAL(10,2)

SELECT * FROM AirportEmployees

```
mysql> SELECT * FROM Airportemployees;
```

employeeID	name	position	department	shift	salary	overtime
1	Ahmed Ali	Security	Security	Day	4500.00	200.00
2	Abdulaziz Omr	Pilot	Flight Crew	Night	90000.00	200.00
3	Mishel Fahd	Air Traffic Controller	Air Traffic Control	Rotating	60000.00	200.00
4	Turki Mohammed	Customer Service Agent	Passenger Services	Evening	4000.00	200.00
5	Walied Talal	Baggage Handler	Ground Operations	Morning	3500.00	200.00
6	Mohammed Fahd	Airport Manager	Administration	Day	10000.00	200.00
7	Jamel Hassn	Maintenance Worker	Facilities	Rotating	55000.00	200.00

```
7 rows in set (0.00 sec)
```

SELECT *FROM AirportEmployees


```
mysql> alter table AirportEmployees add total_sal DECIMAL(10, 2);
Query OK, 0 rows affected (0.03 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Airportemployees;
```

employeeID	name	position	department	shift	salary	overtime	total_sal
1	Ahmed Ali	Security	Security	Day	4500.00	200.00	NULL
2	Abdulaziz Omr	Pilot	Flight Crew	Night	90000.00	200.00	NULL
3	Mishel Fahd	Air Traffic Controller	Air Traffic Control	Rotating	60000.00	200.00	NULL
4	Turki Mohammed	Customer Service Agent	Passenger Services	Evening	4000.00	200.00	NULL
5	Walied Talal	Baggage Handler	Ground Operations	Morning	3500.00	200.00	NULL
6	Mohammed Fahd	Airport Manager	Administration	Day	10000.00	200.00	NULL
7	Jamel Hassn	Maintenance Worker	Facilities	Rotating	55000.00	200.00	NULL

```
7 rows in set (0.00 sec)
```

Alter table AirportEmployees add total_sal DECIMAL

SELECT * FROM AirportEmployees

```
mysql> update AirportEmployees set total_sal = salary + overtime;
Query OK, 7 rows affected (0.00 sec)
Rows matched: 7 Changed: 7 Warnings: 0

mysql> SELECT * FROM Airportemployees;
```

employeeID	name	position	department	shift	salary	overtime	total_sal
1	Ahmed Ali	Security	Security	Day	4500.00	200.00	4700.00
2	Abdulaziz Omr	Pilot	Flight Crew	Night	90000.00	200.00	90200.00
3	Mishel Fahd	Air Traffic Controller	Air Traffic Control	Rotating	60000.00	200.00	60200.00
4	Turki Mohammed	Customer Service Agent	Passenger Services	Evening	4000.00	200.00	4200.00
5	Walied Talal	Baggage Handler	Ground Operations	Morning	3500.00	200.00	3700.00
6	Mohammed Fahd	Airport Manager	Administration	Day	10000.00	200.00	10200.00
7	Jamel Hassn	Maintenance Worker	Facilities	Rotating	55000.00	200.00	55200.00

```
7 rows in set (0.00 sec)
```

Update AirportEmployees set total_sal=salary+overtime

SELECT * FROM AirportEmployees

```
mysql> select BookingNo,BookingStatus from RESERVATIONS;
```

```
+-----+-----+
| BookingNo | BookingStatus |
+-----+-----+
| B0001     | Confirmed     |
| B0002     | Pending       |
| B0003     | Cancelled     |
| B0004     | Completed     |
| B0005     | Pending       |
| B0006     | Confirmed     |
| B0007     | Cancelled     |
+-----+-----+
7 rows in set (0.00 sec)
```

SELECT BookingNo,BookingStatus from RESERVATIONS

```
mysql> Alter table Travelers rename column TravelerNO to TravelerID;
Query OK, 0 rows affected (0.02 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM Travelers;
```

```
+-----+-----+-----+-----+-----+-----+-----+-----+
| TravelerID | bookingNO | FirstName | LastName | PassportNumber | DateOfBirth | Nationality | PhoneNumber |
+-----+-----+-----+-----+-----+-----+-----+-----+
| T101       | B0001     | Rose      | Nife     | AB123456       | 1990-01-22 | SAUDI      | 0539847599 |
| T102       | B0002     | Rana      | Abdullah | RA1212156      | 1999-10-12 | SAUDI      | 0567823465 |
| T103       | B0003     | Sara      | Saad     | SA983456       | 2000-02-15 | SAUDI      | 0587234566 |
| T104       | B0004     | Mada      | Jamel    | MA116836       | 1996-07-11 | SAUDI      | 0599821345 |
| T105       | B0005     | Rahaf     | Ali      | RJ987345       | 2002-06-29 | SAUDI      | 058763211234 |
| T106       | B0006     | Jana      | Mohamed  | CD789012       | 2001-08-25 | SAUDI      | 0598773214 |
| T107       | B0007     | Naif      | Ahmed    | FG182903       | 1997-04-23 | SAUDI      | 0518254930 |
+-----+-----+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

Alter table travelers rename column travelerNO to travelID

SELECT *from traveler

```

mysql> alter table Travelers add title varchar(30);
Query OK, 0 rows affected (0.04 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> UPDATE Travelers
    -> SET title = 'Miss. '
    -> WHERE firstname IN ('Rose', 'Rahaf');
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql>
mysql> UPDATE Travelers
    -> SET title = 'Mrs. '
    -> WHERE firstname IN ('Rana', 'Sara');
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0

mysql>
mysql> UPDATE Travelers
    -> SET title = 'Mr. '
    -> WHERE firstname = 'Naif';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE Travelers
    -> SET title = 'Ms. '
    -> WHERE firstname IN ('Mada', 'Jana');
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0

```

alter table travelers add title varchar(30)

.UPDATE travelers

. UPDATE travelers

. UPDATE travelers

. UPDATE travelers

```
mysql> SELECT * FROM Travelers;
```

TravelerID	bookingNO	FirstName	LastName	PassportNumber	DateOfBirth	Nationality	PhoneNumber	title
T101	B0001	Rose	Nife	AB123456	1990-01-22	SAUDI	0539847599	Miss.
T102	B0002	Rana	Abdullah	RA1212156	1999-10-12	SAUDI	0567823465	Mrs.
T103	B0003	Sara	Saad	SA983456	2000-02-15	SAUDI	0587234566	Mrs.
T104	B0004	Mada	Jamel	MA116836	1996-07-11	SAUDI	0599821345	Ms.
T105	B0005	Rahaf	Ali	RJ987345	2002-06-29	SAUDI	058763211234	Miss.
T106	B0006	Jana	Mohamed	CD789012	2001-08-25	SAUDI	0598773214	Ms.
T107	B0007	Naif	Ahmed	FG182903	1997-04-23	SAUDI	0518254930	Mr.

```
7 rows in set (0.00 sec)
```

.SELECT *from Travelers

```
mysql> update AirportEmployees set salary=4200 where name='Turki Mohammed';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT * FROM AirportEmployees;
```

employeeID	name	position	department	shift	salary	overtime	total_sal
1	Ahmed Ali	Security	Security	Day	4500.00	200.00	4700.00
2	Abdulaziz Omr	Pilot	Flight Crew	Night	90000.00	200.00	90200.00
3	Mishel Fahd	Air Traffic Controller	Air Traffic Control	Rotating	60000.00	200.00	60200.00
4	Turki Mohammed	Customer Service Agent	Passenger Services	Evening	4200.00	200.00	4200.00
5	Walied Talal	Baggage Handler	Ground Operations	Morning	3500.00	200.00	3700.00
6	Mohammed Fahd	Airport Manager	Administration	Day	10000.00	200.00	10200.00
7	Jamel Hassn	Maintenance Worker	Facilities	Rotating	55000.00	200.00	55200.00

```
7 rows in set (0.00 sec)
```

update AirportEmployees set salary=4200 where name ='Turki Mohammed'

SELECT*from AirportEmployees

```
mysql> delete from flights where FlightNo = "L9W3RG";
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM flights;
```

FlightNo	AirplaneID	AirplaneType	DepartmentTime	ArrivalTime	FlightDuration	DestinationAirport
DK8SM4	802	Airbus	13:30:00.00	16:30:00.00	3 Hours	DMM
OD3VY7	807	Airbus	17:20:00.00	19:30:00.00	2 Hours	AUH
OW2V9M	805	Airbus	11:50:00.00	19:05:00.00	6 Hours	CAI
P7BPTV	801	Airbus	16:55:00.00	18:55:00.00	2 Hours	JED
PWM4XC	804	Airbus	26:40:00.00	18:55:00.00	2 Hours	DOH
Q7V3SK	806	Airbus	11:00:00.00	15:10:00.00	4 Hours	RUH

```
6 rows in set (0.00 sec)
```

delete from flights where FlightNo = "L9W3RG"

SELECT*from flights

```
mysql> UPDATE Travelers SET PassportNumber = 'RA121215' WHERE firstname = 'Rana';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> UPDATE Travelers SET PhoneNumber = '0587632112' WHERE firstname = 'Rahaf';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

```
mysql> SELECT * FROM travelers;
```

TravelerID	bookingNO	FirstName	LastName	PassportNumber	DateOfBirth	Nationality	PhoneNumber	title
T101	B0001	Rose	Nife	AB123456	1990-01-22	SAUDI	0539847599	Miss.
T102	B0002	Rana	Abdullah	RA121215	1999-10-12	SAUDI	0567823465	Mrs.
T103	B0003	Sara	Saad	SA983456	2000-02-15	SAUDI	0587234566	Mrs.
T104	B0004	Mada	Jamel	MA116836	1996-07-11	SAUDI	0599821345	Ms.
T105	B0005	Rahaf	Ali	RJ987345	2002-06-29	SAUDI	0587632112	Miss.
T106	B0006	Jana	Mohamed	CD789012	2001-08-25	SAUDI	0598773214	Ms.
T107	B0007	Naif	Ahmed	FG182903	1997-04-23	SAUDI	0518254930	Mr.

```
7 rows in set (0.00 sec)
```

**UPDATE Travelers SET PassportNumber = 'RA121215 '
WHERE first name ='RANA'**

**UPDATE Travelers SET PhoneNumber = '0587632112 '
WHERE first name ='RAHAF'**

SELECT*from travelers

The End .